



Java

Clase 03 | Cadenas y listas

Índice

Clase 03

Clase 03 | Cadenas y listas

- Manipulación de cadenas de caracteres.
- Métodos comunes de cadenas.
- Introducción a arrays y listas.
- Uso de ArrayList y sus métodos.

Clase 04

Clase 4 | Funciones y Modularización

- Concepto de funciones (métodos) en Java.
- Declaración y llamada de funciones.
- Parámetros y argumentos.
- Paso por valor y referencia.
- Sobrecarga de métodos.
- Buenas prácticas de modularización.

Objetivos de la Clase.



Comprender cadenas de caracteres en Java

Aprenderemos las principales operaciones para manipular strings.



Introducir arrays y sus limitaciones

Exploraremos el uso de arrays para almacenar colecciones de datos.



Conocer listas dinámicas (ArrayList)

Descubriremos cómo manejar colecciones flexibles con ArrayList.



Aplicar conocimientos en TechLab

Resolveremos problemas concretos del catálogo de productos.

Manejo de Strings

Manipulación de cadenas de caracteres

Las cadenas de caracteres (Strings) son uno de los tipos de datos más importantes y utilizados en Java. Para trabajar con ellas de manera efectiva, es fundamental entender sus características y métodos principales.

Un aspecto crucial de las cadenas en Java es su inmutabilidad. Esto significa que una vez creada una cadena, su contenido **no puede modificarse**.

¿Por qué es importante?

- Garantiza seguridad en aplicaciones multihilo
- Mejora el rendimiento del pool de strings
- Previene modificaciones accidentales



Ejemplo: Inmutabilidad

```
String nombre = "Juan";  
nombre.toUpperCase();  
// nombre sigue siendo "Juan"
```

```
nombre = nombre.toUpperCase();  
// Ahora sí, nombre es "JUAN"
```

El método `.toUpperCase()` devuelve una nueva cadena en mayúsculas, pero no modifica el objeto original. Como no asignas el resultado a una variable, el nuevo objeto "JUAN" creado es descartado, y `nombre` sigue apuntando al `String` original "Juan".

Ahora, el resultado del método `.toUpperCase()` (es decir, el nuevo objeto "JUAN") se asigna a la variable `nombre`. Esto hace que `nombre` apunte al nuevo objeto, mientras que el objeto original "Juan" permanece sin cambios (aunque ya no es accesible desde esta variable).

Ejemplo: formateo de nombres de productos

Este proceso elimina espacios extras y estandariza las mayúsculas/minúsculas.

```
String producto = " Café Premium MOLIDO ";  
producto = producto.trim().toLowerCase();  
// Resultado: "café premium molido"
```

Formateo avanzado

Este método procesa nombres de productos aplicando las siguientes transformaciones:

Entrada: Recibe un String que puede contener espacios extras y mezcla de mayúsculas/minúsculas.

Salida: Devuelve un String donde cada palabra comienza con mayúscula y el resto en minúscula (formato Title Case).

Ejemplo:

- Entrada: "caFÉ PREMIUM molido"
- Salida: "Café Premium Molido"

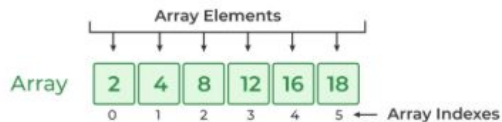
```
public static String formatearNombreProducto(String nombre) {  
    nombre = nombre.trim().toLowerCase();  
    String[] palabras = nombre.split(" ");  
    StringBuilder sb = new StringBuilder();  
    for (String palabra : palabras) {  
        if (!palabra.isEmpty()) {  
  
            sb.append(Character.toUpperCase(palabra.charAt(0)))  
              .append(palabra.substring(1)).append(" ");  
        }  
    }  
    return sb.toString().trim();  
}
```


Arrays

Introducción a Arrays

Los arrays son una estructura de datos fundamental en Java que nos permite almacenar múltiples elementos del mismo tipo en una única variable. Se declara con un tipo, seguido de [], por ejemplo:

```
String[] productos = new String[3];  
productos[0] = "Café Premium Molido";  
productos[1] = "Té Verde Orgánico";  
productos[2] = "Chocolate Amargo 80%";
```



Características de los Arrays

1

Estructura de Datos Fija

Los arrays tienen un tamaño fijo una vez creados. Por ejemplo, si creamos un array de 5 elementos, este tamaño no podrá modificarse durante la ejecución del programa.

2

Declaración y Creación

Se declaran con un tipo seguido de [], ejemplo: `String[]`. La creación puede ser directa o mediante `new`.



Comprender estas características es esencial para decidir cuándo usar arrays y cuándo considerar estructuras de datos más flexibles como `ArrayList`.

Ejemplo de Array.

Este código muestra cómo crear y recorrer un array de productos.

```
String[] productos = new String[3];  
productos[0] = "Café Premium Molido";  
productos[1] = "Té Verde Orgánico";  
productos[2] = "Chocolate Amargo 80%";  
  
for (int i = 0; i < productos.length; i++) {  
    System.out.println("Producto: " +  
        productos[i]);  
}
```

1. Se declara un arreglo de `String` llamado `productos` con 3 posiciones.
2. Se asignan nombres de productos a las posiciones del arreglo.
3. Se usa un bucle `for` para recorrer el arreglo e imprimir cada producto.

Salida esperada:

```
Producto: Café Premium Molido  
Producto: Té Verde Orgánico  
Producto: Chocolate Amargo 80%
```

ArrayList

Introducción a ArrayList.

ArrayList es una lista dinámica que crece o decrece según necesidad.

1

Métodos Útiles

Ofrece add(), remove(), get() para manejar elementos fácilmente.

2

Sin Preocupación por Tamaño

No es necesario especificar un tamaño fijo al crear ArrayList.

3

Tipo Genérico

Se declara con un tipo específico, por ejemplo: ArrayList.

Ejemplo de ArrayList.

```
ArrayList listaProductos = new ArrayList<>();  
listaProductos.add("Café Premium Molido");  
listaProductos.add("Té Verde Orgánico");  
listaProductos.add("Chocolate Amargo 80%");  
  
for (String producto : listaProductos) {  
    System.out.println(producto);  
}
```

Este código utiliza una `ArrayList` para manejar una lista dinámica de productos y es una alternativa más flexible a los arreglos tradicionales en Java. Aquí tienes un análisis resumido.

- ❗ **Ventaja principal:** `ArrayList` es más flexible que los arreglos tradicionales, ya que tiene un tamaño dinámico y métodos útiles para la manipulación de datos.

Métodos Comunes de ArrayList

add(elemento)
Agrega un elemento al final de la lista.



get(índice)
Obtiene el elemento en la posición dada.



remove(índice/objeto)
Elimina un elemento por índice o por objeto.



size()
Retorna la cantidad de elementos en la lista.



Ejemplo práctico

● Problema: Actualización dinámica del catálogo

⚠ Se necesita añadir un nuevo producto "Café Descafeinado" al catálogo digital debido a la creciente demanda de los clientes y se decide que "Chocolate Amargo 80%" ya no estará disponible

► [Hacé clic acá para ver la solución](#)

▼ [Hacé clic acá para ver la solución](#)

```
ArrayList listaProductos = new ArrayList<>();
listaProductos.add("Café Premium Molido");
listaProductos.add("Té Verde Orgánico");
listaProductos.add("Chocolate Amargo 80%");

// Agregar nuevo producto
listaProductos.add("Café Descafeinado");

// Eliminar producto
listaProductos.remove("Chocolate Amargo 80%");

// Verificar existencia
if (listaProductos.contains("Té Verde Orgánico")) {
    System.out.println("El producto existe");
}
```

Recursos Adicionales



Documentación Oficial de Java

Consulta la documentación de String y ArrayList para profundizar.



docs.oracle.com



ArrayList (Java Platform ...

The size, isEmpty, get, set, iterator, and listIterator operations run in constant...



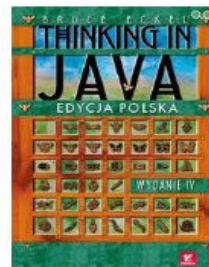
Tutoriales en YouTube

Explora tutoriales recomendados sobre manipulación de cadenas y colecciones.



Libros de Programación

Amplía tus conocimientos con libros especializados en Java.



Reflexiones Finales

Inmutabilidad de Strings

¿Por qué son inmutables las cadenas en Java?

Arrays vs ArrayList

¿Cuándo conviene usar cada uno?

Impacto en Proyectos

¿Cómo contribuye esto a la escalabilidad de TechLab?

Aplicaciones Prácticas

¿Qué otros usos ves para estas herramientas?

Ejercicios prácticos

⚙️ Situación inicial en TechLab.



Silvia, la Product Owner, tiene en el **backlog*** una serie de pedidos del cliente:

- Los nombres de algunos productos vienen con espacios extra o en mayúsculas y minúsculas inconsistentes. Es necesario limpiar y formatear esos nombres antes de mostrarlos al cliente.
- El catálogo de productos empieza a crecer, y almacenar los nombres en variables sueltas ya no es práctico. Necesitan una estructura que permita agruparlos. Primero probarán con arrays, pero pronto notarán las limitaciones.
- Finalmente, para agregar y quitar productos fácilmente del catálogo, un ArrayList será la solución. Además, Sabrina y Matías quieren filtrar, buscar y modificar productos de forma dinámica.

Ejercicio práctico

Optativo | No entregable

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

Manipulación de Cadenas:

- Dada una cadena " té CHAi ", formateala para que quede "Té Chai".
- Mostrá su longitud, su primera letra y verificá si contiene la palabra "Chai".

Arrays:

- Creá un array de 5 productos.
- Imprimí sus elementos y luego intentá agregar un sexto producto (analizá el resultado).

Ejercicio práctico

Optativo | No entregable

ArrayList:

- Creá un `ArrayList<String>` para productos.
- Agregá varios productos, eliminá uno, verificá si otro existe y luego imprimí la lista final.

Combinar Cadenas y Listas:

- Tené una lista de productos con nombres desprolijos.
- Creá un método que recorra la lista y aplique el formateo a cada nombre, volviendo a imprimir la lista con nombres prolijos.